

GRANNY

Master Build Blueprint

AI-Based Cognitive Gaming & Memory Assistance for the Elderly — SIH26003

*Phase-wise development roadmap · Complete feature specification ·
Ready-to-use master prompts for Backend, AI Services, Web, Mobile & the 10-Game World*



Contents

#	Section
1	Project Snapshot & Guiding Principle
2	Complete Feature Specification
3	System Architecture Recap
4	Phase-Wise Development Roadmap
5	Database Schema Blueprint
6	Master Prompt — Backend (NestJS)
7	Master Prompt — AI Microservice (FastAPI)
8	Master Prompt — The 10-Game Memory World (Game Engine)
9	Master Prompt — Web Frontend (React)
10	Master Prompt — Mobile App (React Native / Expo)
11	Safety, Ethics & Accessibility Checklist
12	SIH Demo & Pitch Checklist

This document is written so each section can be copy-pasted directly into an AI coding assistant (Claude Code, Cursor, etc.) or handed to a teammate as a standalone spec. Build top-to-bottom — each phase assumes the previous one is done.

Project Snapshot & Guiding Principle

Core idea: “Don’t make the elderly adapt to the game. Make the game adapt to the elderly.” Granny is not 10 separate mini-games bolted onto an app — it is **one continuous, voice-first world** (Home → Market → Journey → Music → Story → Garden → Album) wrapped around a memory-assistant companion that quietly tracks accuracy, response time, mistakes and preferences, and reshapes future sessions around them.

What makes this different from a generic “brain training app”

- **Identity-preserving, not generic:** games are populated with the user's own rooms, routines and life-story, not stock puzzles.
- **Voice-first, not tap-first:** speech is the primary input; large-touch UI is the fallback, not the other way round.
- **Adaptive by design:** a single difficulty engine drives every game and every companion interaction, not per-game tuning.
- **Family-in-the-loop:** caregivers get visibility and light co-play, without turning the app into a surveillance tool.
- **Safety-first:** distress detection and gentle escalation are architectural, not an afterthought bolted on later.

Complete Feature Specification

Features are grouped into **Core (MVP — build first, demoable for SIH)**, **Companion Intelligence**, and **Stretch / Differentiators** pulled from your research notes (memory theatre, SAR companion, JIT micro-interventions, family co-play, culturally adaptive tone).

2.1 Core MVP Features

Feature	What it does
Cognitive Game World	All 10 games (Remember My Home, Memory Market, Name & Face Match, Recipe Recall, Memory Journey, Complete the Tune, Story Detective, Where Did I Keep It, Memory Garden, Memory Album) linked as one scrollable/explorable world.
Adaptive Difficulty Engine	Central service scores accuracy, latency and error type per attempt; raises/lowers item count, delay time and distractor count per user, per game.
Voice Interaction	STT → intent parsing → empathetic LLM response → TTS, with natural pacing and barge-in support.
Personal Memory Assistant	Vector-store of the user's own facts (names, routines, past sessions) retrieved before every response for contextual recall.
Daily Routine & Medication Reminders	Scheduled, voice + push reminders; confirmation logging; non-diagnostic phrasing only.
Family Circle Dashboard	Caregivers see adherence, mood trend and game performance; can add memories/photos with consent.
Safety Guardrails	Distress-phrase detection → de-escalation script → caregiver/emergency-contact alert.
Elderly-First Accessibility	High contrast, 48px+ touch targets, large text, single-column layouts, no nested menus.

2.2 Companion Intelligence Layer

Feature	What it does
Emotion & Tone Awareness	Acoustic + linguistic sentiment analysis (happy / sad / confused / distressed / calm) reshapes response tone and pacing in real time.
Conversational Continuity	Session-to-session memory of what was discussed so the companion never “forgets” yesterday’s chat.
Voice Recognition per Family Member	Enrolled voiceprints let Granny greet “Raj” differently from “Amma” and tailor the relationship.
Multilingual / Regional Speech	English at launch, with an i18n layer designed from day one for regional language packs.

2.3 Stretch / Differentiator Features (from research)

Feature	What it does
Life-Story Memory Theatre	LLM turns the elder's own recorded anecdotes into short narrated scenes or choose-your-own-path "memory episodes," scaffolded by cognitive level (MMSE-style).
Just-in-Time Micro-Interventions	1–3 minute cognitive "micro-doses" triggered by context (post-lunch, post-nap, idle time) rather than fixed sessions.
Family Co-Play Memory Quests	AI generates a quest from real family photos/events; relatives join remotely and the AI balances difficulty so the elder leads, not struggles.
Culturally Adaptive Companion	Content (festivals, songs, stories) and companion tone adapt to the user's stated cultural/regional context.
Socially Assistive Robot / Avatar Mode	Same backend, future embodiment: a robot or animated avatar delivers the same adaptive tasks inside daily routines.

Recommendation: build 2.1 fully for SIH demo day, wire up a thin slice of 2.2 (emotion tone on one flow), and pitch 2.3 as the roadmap slide — judges reward vision as much as working code.

3

System Architecture Recap

Four layers, already decided in your README — this document builds directly on top of it.

Layer	Responsibility
Clients	React (Vite) web portal + React Native (Expo) mobile app — both elderly-optimized, both talk to the same backend.
Backend (NestJS)	REST + WebSocket gateway: auth, users, games state, family/caregiver module, reminders, notifications. Owns PostgreSQL via Prisma.
AI Microservice (FastAPI)	Speech-to-text (Whisper), text-to-speech, the empathetic LLM persona + safety guardrails, semantic memory/embeddings, emotion analyzer.
Data & Messaging	PostgreSQL (source of truth), Redis (sessions, queues, cache), Firebase Cloud Messaging (push for meds/family alerts).

Contract between layers: the NestJS backend never calls the LLM directly for conversation — it proxies to the FastAPI service so all AI logic, prompts and safety rules live in one place and can be iterated on without redeploying the core API.

Phase-Wise Development Roadmap

Thirteen phases, sequenced so you always have something demoable. Durations assume a small SIH team (3–5 people) working part-time; compress by running Phases 3–4 and 6–8 in parallel across two sub-teams once the backend contract is frozen.

Ph	Focus	Key Deliverables	Est.
0	Research & Requirements Lock	Finalize personas, the 10-game world map, MVP vs stretch list (Section 2), wireframe the Home→Album flow on paper.	3–4 days
1	Repo, Env & DevOps Setup	Monorepo scaffold (apps/web, apps/mobile, backend, ai-services, packages), Docker Compose for Postgres+Redis, CI skeleton, .env templates.	2–3 days
2	Database & Backend Core	Prisma schema (Section 5), Auth module (elder + caregiver roles), Users/Profiles, JWT/session, base REST scaffolding for every module.	1–1.5 wks
3	AI Microservice Core	FastAPI skeleton, STT endpoint (Whisper), TTS endpoint, a single /converse endpoint wired to an LLM with the persona + safety system prompt.	1–1.5 wks
4	Semantic Memory Engine	Vector store (pgvector or a managed vector DB), embedding pipeline for facts/anecdotes, retrieval-augmented context injection into /converse.	1 wk
5	Adaptive Difficulty Engine	Central scoring service: accuracy, latency, error taxonomy → next-difficulty function; exposed as one API every game calls.	4–6 days
6	Build the 10 Games	Implement each game (Section 8) against the shared Game Engine contract: start_session, submit_attempt, get_next_item, end_session.	2–3 wks
7	Web Frontend Shell + Voice	React app shell, accessibility design system, voice capture/playback, wire login → Home world → first 3 games end-to-end.	1.5–2 wks
8	Web Frontend — Remaining Games + Companion Chat	Remaining 7 games, chat/companion screen, memory album uploader, routine/medication reminder UI.	1.5–2 wks
9	Family/Caregiver Dashboard	Caregiver web views: adherence, mood trend, game performance charts, consent-gated memory editing, invite flow.	1 wk
10	Emotion & Safety Layer	Emotion analyzer wired into /converse, distress-phrase detector, de-escalation script, caregiver/emergency alert pipeline (FCM).	1 wk
11	Mobile App (Expo)	Port the accessible design system to React Native, reuse the api-client package, prioritize voice + reminders + one flagship game.	1.5–2 wks
12	Testing, Accessibility Audit & Deploy	Elderly usability testing (5–8 real users if possible), WCAG contrast/touch-target audit, load test AI endpoints, deploy + demo script.	1 wk

Suggested SIH cut-line: if time runs out, everything through Phase 8 plus a slice of Phase 10 (one safety keyword demo) is a complete, judgeable product. Phases 9, 11, 12 are what separate a finalist from a winner.

Minimum viable Prisma schema to unblock Phase 2. Extend, don't restructure, as features land.

```
model User {
  id          String   @id @default(uuid())
  role        Role     @default(ELDER)    // ELDER | CAREGIVER
  name        String
  phone       String?  @unique
  language    String   @default("en")
  familyGroupId String?
  createdAt   DateTime @default(now())
  profile     ElderProfile?
  gameSessions GameSession[]
  memories    Memory[]
  conversations Conversation[]
}

model ElderProfile {
  id          String   @id @default(uuid())
  userId      String   @unique
  ageGroup    String?
  interests   String[]
  cognitiveLvl Int      @default(3)      // 1-5, drives base difficulty
  culturalTags String[]
}

model Memory {
  id          String   @id @default(uuid())
  userId      String
  type        String   // fact | anecdote | photo | routine
  content     String
  embedding    Unsupported("vector(1536)")?
  addedBy     String   // ELDER | CAREGIVER | SYSTEM
  createdAt   DateTime @default(now())
}
```

```

model Game { id String @id @default(uuid()) key String @unique name String }

model GameSession {
  id          String    @id @default(uuid())
  userId      String
  gameKey     String
  difficulty  Int
  startedAt   DateTime @default(now())
  endedAt     DateTime?
  attempts    Attempt[]
}

model Attempt {
  id          String    @id @default(uuid())
  sessionId   String
  itemIndex   Int
  correct     Boolean
  latencyMs   Int
  errorType   String?
}

model Conversation {
  id          String    @id @default(uuid())
  userId      String
  role        String    // user | assistant
  text        String
  emotion     String?
  createdAt   DateTime @default(now())
}

model Reminder {
  id          String    @id @default(uuid())
  userId      String
  type        String    // medication | hydration | activity
  scheduleCron String
  lastConfirmedAt DateTime?
}

```

Master Prompt — Backend (NestJS)

Paste this into your AI coding assistant once Phases 0–1 are done and the schema in Section 5 is agreed. Fill the bracketed notes before sending.

Backend master prompt

```
You are building the NestJS backend for "Granny", an AI cognitive-gaming and
memory-assistance app for elderly users. Stack: NestJS + TypeScript, Prisma ORM,
PostgreSQL, Redis (BullMQ for queues), WebSockets (Socket.IO gateway), Firebase
Admin SDK for push. This service NEVER calls the LLM directly – it proxies
conversation/voice calls to an internal FastAPI service at AI_SERVICE_URL.

Build these modules, each with controller + service + DTOs + Prisma repository:
1. auth          - phone/OTP + email login, JWT, roles ELDER/CAREGIVER, refresh tokens
2. users          - profile CRUD, family-group linking, consent flags
3. memory         - CRUD for Memory records; POST forwards new text/photo memories
                  to the AI service for embedding, then stores the returned vector reference
4. games          - game catalog, start/end session, record attempts, fetch the
                  next item's difficulty by calling the AI service's /difficulty/next endpoint
5. conversation   - proxies voice/text turns to AI service /converse, persists
                  the exchange + emotion label returned
6. reminders      - CRUD schedules, BullMQ cron jobs that fire FCM pushes + a
                  WebSocket event the client turns into a spoken reminder
7. family         - caregiver dashboard read endpoints: adherence %, mood trend,
                  last N game sessions, pending consent requests
8. safety         - receives distress flags from the AI service, fans out FCM
                  alerts to the family group, logs an incident record

Cross-cutting requirements:
- Every ELDER-scoped endpoint must verify the caller owns the resource or is a
  linked CAREGIVER with explicit consent – write a reusable OwnershipGuard.
- All list endpoints paginated; all mutation endpoints validated with class-validator.
- Structured logging (pino) with request IDs; no PII in logs.
- Health check endpoint /health for Docker Compose readiness probes.
- Write Prisma schema exactly as in the provided blueprint, then generate migrations.
- Provide a docker-compose service block and a .env.example with every variable used.

Deliverable: a working NestJS app that boots with `docker-compose up`, exposes
Swagger docs at /docs, and passes a Jest test per module covering the happy path
and the ownership/consent guard rejection path.
```

AI microservice master prompt

You are building the Python AI microservice for "Granny". Stack: FastAPI, Pydantic v2, an LLM provider behind a thin adapter interface (so it can be swapped), a vector store (pgvector via SQLAlchemy, or a managed vector DB), a Whisper-based STT pipeline, and a TTS engine behind a similar adapter.

Endpoints to build:

1. POST /stt audio bytes -> transcript + language + confidence
2. POST /tts text + voice profile -> audio stream, natural pacing
3. POST /converse { userId, text, sessionHistory } ->
 - a) retrieve top-k relevant Memory embeddings for this userId
 - b) run the emotion analyzer on the input text (+ optional audio features)
 - c) build the system prompt from: persona template + retrieved memories + emotion state + safety rules (see safety notes below)
 - d) call the LLM, return { reply, emotion, distressFlag, suggestedAction }
4. POST /memory/embed text or image caption -> embedding vector, stored with metadata (userId, type, addedBy, timestamp)
5. POST /memory/search userId + query -> top-k relevant memories (used by /converse internally and exposed for the family dashboard's search)
6. POST /difficulty/next { userId, gameKey, recentAttempts[] } -> next difficulty parameters (itemCount, delaySeconds, distractorCount) using an exponential moving average of accuracy + latency, clamped to 5 levels
7. POST /life-story/generate (stretch) anecdote text -> a short narrated scene or 3-step choose-your-path script, scaffolded by cognitiveLvl

Persona & safety rules (bake into the system prompt template, not per-call):

- Warm, patient, never condescending; short sentences; no jargon.
- Never gives medical diagnoses or changes medication guidance – redirect to a family member or professional for anything clinical.
- Distress-phrase detector runs BEFORE the LLM call; on a match, skip normal generation and return a fixed gentle de-escalation script + distressFlag=true.
- Never invents facts about the user – only uses retrieved Memory records.

Non-functional requirements:

- Async endpoints throughout; STT/TTS calls streamed where the provider allows.
- Config via pydantic-settings reading from .env; no secrets in code.
- Structured JSON logging; redact transcript text at INFO level, full at DEBUG only.
- Dockerfile + inclusion in the shared docker-compose.yml on port 8000.
- Pytest coverage for the difficulty function and the distress-phrase detector, since those two are safety- and pedagogy-critical and must not regress silently.

Master Prompt — The 10-Game Memory World (Game Engine)

All 10 games share one contract so the Adaptive Difficulty Engine and the UI shell only need to be built once. Use this prompt after Phases 2, 3 and 5 exist.

Game engine master prompt

You are building the shared Game Engine contract and all 10 cognitive games for "Granny", to run inside the React web app (and later React Native). Games are content, not separate engines – implement ONE TypeScript interface and 10 content modules that satisfy it.

```
interface CognitiveGame {
  key: string;
  title: string;
  primarySkills: string[]; // e.g. ['spatial-memory', 'visual-memory']
  startSession(userId, difficulty): SessionState;
  getNextItem(sessionState): GameItem; // question/prompt + choices/targets
  submitAttempt(sessionState, response): AttemptResult; // correct, latencyMs, errorType
  endSession(sessionState): SessionSummary;
}
```

Implement all 10 using this contract, each in its own content module:

1. Remember My Home - spatial+visual; place N objects in a room scene, hide, ask elder to point to where each object was.
2. Memory Market - working memory+attention; show a 2-5 item shopping list, elder finds matching items in a virtual aisle grid.
3. Name & Face Match - recognition+associative; introduce K character cards, later re-show faces and ask elder to say or pick the name.
4. Recipe Recall - sequential+executive; show a step sequence, shuffle, elder reorders via drag or voice ("first we wash, then...").
5. Memory Journey - episodic+sequential; a guided "walk" through N locations, then ask what was seen and in what order.
6. Complete the Tune - auditory+associative; play a clip, elder names it or hums/says the next line (speech match against a phonetic reference).
7. Story Detective - auditory+comprehension; TTS narrates a short story, then asks 2-3 comprehension questions.
8. Where Did I Keep It - working+spatial; everyday objects placed around a home scene, delayed recall of locations.
9. Memory Garden - visual+spatial+long-term; persistent user-built garden, later sessions test recall of prior placements (cross-session state).
10. Memory Album - recognition+episodic; show a mix of seen/unseen images, elder identifies which they've seen and recalls details.

Shared requirements:

- Every game calls POST /difficulty/next after each session to get the next itemCount/delaySeconds/distractorCount – never hardcode difficulty curves.
- Every submitAttempt logs { correct, latencyMs, errorType } via the games module so the backend can persist Attempt rows.
- Voice-first: every prompt has a TTS narration and accepts a spoken response via STT, with large-button tap as a fallback for every interaction.
- Games 5, 9 and 10 must read from the user's own Memory records (real family photos, real places named by the caregiver) rather than stock content when available.
- Provide a GameWorldMap component: a single scrollable/explorable canvas that visually links Home -> Market -> Journey -> Music -> Story -> Garden -> Album as connected "rooms", not a grid of app icons.

Deliverable: the shared interface, all 10 content modules, unit tests for each game's getNextItem/submitAttempt logic, and Storybook (or equivalent) entries so each game can be visually reviewed in isolation.

Web frontend master prompt

You are building the React web app for "Granny". Stack: React 18 + Vite + TypeScript + Tailwind CSS + Zustand for state + the shared api-client package. Audience: elderly users AND their caregivers, via role-based routing.

Design system constraints (non-negotiable, elderly-first accessibility):

- Minimum touch target 48x48px; primary actions 64px+.
- Base font size 20px+, high-contrast palette (WCAG AA minimum, aim for AAA on primary text), no light-grey-on-white text anywhere.
- Single-column layouts, max one primary action per screen, no hover-only affordances, no auto-advancing carousels.
- Every screen reachable within 2 taps/voice-commands from Home.
- A persistent large mic button is the primary way to interact; typed input is a secondary fallback, never the only path.

Screens to build, in this order:

1. Auth (phone/OTP) -> role-aware redirect (Elder Home vs Caregiver Dashboard)
2. Elder Home / GameWorldMap - the connected Home->Album world (Section 8)
3. Companion Chat - voice-first conversation view with waveform/listening state, large transcript text, gentle typing/thinking indicator
4. Reminders - today's schedule, one-tap "I took it" confirmation, gentle escalation UI if a reminder is repeatedly missed
5. Memory Album manager - upload/tag photos & anecdotes, consent toggle for what the companion is allowed to reference
6. Caregiver Dashboard - adherence chart, mood trend, recent game performance, pending consent/invite actions (reuse chart_display-style components)
7. Settings/Accessibility - text size, voice speed, language, contrast toggle

Technical requirements:

- api-client package wraps every backend call with typed request/response DTOs matching the NestJS DTOs – do not hand-roll fetch calls in components.
- WebSocket connection for live reminder pushes and companion streaming replies.
- Web Audio API for mic capture -> chunked upload to /conversation/voice.
- Route guards for ELDER vs CAREGIVER, redirecting unauthorized roles.
- i18n scaffold (even English-only at launch) so regional languages are a content swap, not a rewrite.

Deliverable: a working Vite app, Storybook for the design-system primitives, and one Cypress/Playwright happy-path test per screen listed above.

Master Prompt — Mobile App (React Native / Expo)

Mobile app master prompt

You are porting "Granny" to React Native (Expo, TypeScript). Reuse the api-client and shared TypeScript types from packages/ – do not duplicate DTOs.

Priority order for mobile (do not try to port everything at once):

1. Voice companion chat (uses expo-av for mic capture, streaming playback)
2. Reminders with native push (expo-notifications + FCM) – this is the single highest-value mobile feature since it works even app-closed
3. ONE flagship game fully playable (recommend 'Where Did I Keep It' – simplest interaction pattern, easiest to validate touch-target sizing on)
4. Elder Home / world map (can reuse most layout logic from web via shared design tokens, but re-implement components natively for gesture quality)
5. Remaining games, family dashboard read-only view

Mobile-specific constraints:

- Respect OS-level accessibility settings (Dynamic Type / font scaling, reduce-motion) rather than fighting them.
- Offline-tolerant reminders: schedule locally via expo-notifications so medication alerts fire even without connectivity; sync confirmation on reconnect.
- Biometric or simple PIN unlock as an alternative to typing a password.
- Test on a physical low-cost Android device, not just iOS simulator – this is the device class most elderly users in India will actually own.

Deliverable: an Expo app that runs on `expo start`, with the reminder flow and one flagship game fully functional end-to-end against the same backend used by the web app.

Before every demo or release, verify:

- The companion never states or implies a medical diagnosis, and never suggests changing a dose or schedule.
- Distress phrases reliably trigger the de-escalation script in a manual test set of at least 20 phrases, including indirect ones.
- Every screen passes a 48px touch-target and WCAG AA contrast check.
- Family/caregiver access to memories and location-like data requires explicit elder or guardian consent, revocable at any time.
- Voice recordings and health-adjacent data (medication logs) are encrypted at rest and in transit.
- No dark patterns: no infinite scroll, no manipulative streaks, no guilt-based reminder copy.
- A visible, one-tap path exists at all times to call a real family member or emergency contact.

SIH Demo & Pitch Checklist

- Open on the problem, in one sentence: elderly users are asked to adapt to technology; Granny adapts to them instead.
- Live-demo the GameWorldMap for 60–90 seconds — walk from Home into 2 games max, don't show all 10.
- Show one adaptive-difficulty moment on screen: a wrong answer visibly softening the next item.
- Show the companion catching a distress phrase and the caregiver alert firing — this is usually the moment that wins judges over.
- Close with the roadmap slide pulling from Section 2.3 (life-story theatre, SAR companion, family co-play quests) to show vision beyond the MVP.
- Have the architecture diagram (Section 3) ready as a backup slide for technical judges.

Built as a working blueprint for the Granny team — SIH 2026 (SIH26003). Treat every “master prompt” section as a living document: update it the moment a real implementation decision diverges from what's written here.